

Projeto MICROC – Etapa 5

Análise Semântica 2: fluxo de controle e dados

2º semestre de 2026

Objetivo

Nesta etapa, o grupo concluirá as verificações semânticas da MICROC. Partindo da AST com nomes resolvidos e tipos verificados, deverá construir um grafo de fluxo de controle (CFG) por função, verificar a inicialização definida das variáveis e impedir que funções não-void alcancem o fim do corpo.

Resultado da etapa

AST com nomes e tipos → CFG auxiliar por função
alcance → inicialização definida → verificação do fim
AST validada ou diagnósticos

O CFG acrescenta conexões entre operações que referenciam a AST existente. A árvore continua sendo a representação principal do compilador e mantém suas posições, vínculos e tipos. A geração de assembly será realizada em uma etapa posterior.

As regras da linguagem permanecem na especificação normativa da MICROC. Este enunciado define o trabalho, as interfaces e os critérios desta entrega.

1 Organização e entrega

1.1 Recupere o projeto anterior

Copie os arquivos Python e os pacotes auxiliares da entrega anterior para a pasta `previous/`. Ela reúne `lexer`, `parser`, `AST`, símbolos, resolução de nomes e verificação de tipos. Os arquivos já fornecidos nessa pasta preservam os contratos e contêm pontos a substituir pelas implementações do grupo.

O arquivo `load_previous.py` disponibiliza esses módulos por seus nomes originais. Não é necessário editar os `imports` antigos ou acrescentar `previous.` a eles. Todas as passagens devem compartilhar as mesmas classes de `AST`, símbolos e tipos.

As versões antigas de `semantic.py`, `runner.py` e `semantic_errors.py` podem acompanhar a cópia dentro de `previous/`. A nova etapa utiliza as versões atualizadas da raiz. Preserve esses arquivos da raiz ao recuperar o trabalho anterior. Não copie diretórios `.git`, ambientes virtuais ou caches.

1.2 Material inicial e entrega

Na raiz, o `starter` contém as estruturas do CFG e sua coordenação inicial em `build`. O grupo implementa o percurso dos blocos da `AST` e o tratamento dos comandos. Também são fornecidos transferências locais, visitantes de expressões, `runner`, exemplos, testes públicos e GitHub Actions.

O grupo deve completar os pontos pendentes:

- `cfg_builder.py`: percurso dos blocos e tratamento dos comandos, incluindo retornos, condicionais, laços e saídas de escopo;
- `reachability.py`: cálculo dos blocos alcançáveis;
- `initialization.py`: junção dos estados e iteração até ponto fixo; e
- `flow_checker.py`: diagnósticos de leitura não inicializada e retorno ausente.

Para distinguir os comandos durante a construção do CFG, use `isinstance` com o objeto do comando e uma classe concreta da `AST`. Essa verificação aceita instâncias da classe indicada e de suas subclasses. Testar somente `Stmt`, a classe base, reconhece todos os comandos, mas não distingue os casos que exigem tratamentos diferentes.

`NotImplementedError` sinaliza esses pontos. Não remova os erros de método pendente sem implementar o comportamento exigido.

Prazo desta etapa

A entrega encerra-se no **domingo, 25 de outubro de 2026, às 23h59**, no horário de Brasília. O commit considerado na correção deve estar no repositório correto do grupo no GitHub até esse horário.

2 Interfaces e metadados

O pipeline completo deve executar nomes, tipos e fluxo nessa ordem:

```
program = Parser(Lexer(source).scan()).parse()
program = SemanticAnalyzer().analyze(program)
```

Em sucesso, `analyze` devolve o mesmo objeto `Program`. `check_flow(program)` retorna `None` em sucesso e lança `SemanticError` se houver diagnósticos de fluxo. Uma passagem com erros impede as seguintes. As verificações de nomes e tipos da entrega anterior continuam obrigatórias, inclusive para comandos inalcançáveis.

`CFGBuilder.build(function)` recebe uma `FunctionDecl` anotada e devolve seu CFG. `compute_reachable(cfg)` devolve o conjunto de IDs alcançáveis a partir da entrada, incluindo a própria entrada.

A passagem de fluxo deve preservar a sintaxe e as anotações `symbol`, `scope` e `type`. Acrescente as anotações seguintes a cada função:

Chave em metadata	Conteúdo
<code>cfg</code>	CFG da função, referenciando a <code>FunctionDecl</code> original.
<code>flow</code>	Objeto <code>FunctionFlowFacts</code> com <code>cfg</code> , <code>reachable</code> e <code>initialization</code> .

`initialization` contém `initialized_in` e `initialized_out`, dicionários de ID de bloco para conjunto de símbolos. Os resultados devem existir para todos os blocos alcançáveis. Números de IDs, quantidade de blocos auxiliares e ordem de diagnósticos independentes não são resultados normativos da linguagem.

3 Construção do CFG

3.1 Estruturas e conexões

Um bloco na AST (`class Block(Stmt)`) delimita um escopo. Um bloco básico do CFG agrupa operações sequenciais e termina com uma decisão de controle. Um único bloco da AST pode produzir vários blocos básicos.

Cada `BasicBlock` possui um `id` e uma lista `operations`. Seu `terminator` define a saída do bloco. Os conjuntos `predecessors` e `successors` registram suas conexões.

As operações guardam referências a declarações, atribuições, comandos de impressão, chamadas como comando e marcas auxiliares `ScopeExit`. Condições e valores retornados ficam nos terminadores correspondentes.

Terminador	Significado
<code>Jump</code>	Continuação incondicional em <code>target</code> .
<code>Branch</code>	Condição <code>condition</code> , com destinos <code>true_target</code> e <code>false_target</code> .
<code>Return</code>	Nó <code>ReturnStmt</code> em <code>statement</code> , com destino na saída por retorno.
<code>Stop</code>	Saída artificial sem sucessores.

O CFG contém `function`, `blocks`, `entry`, `return_exit` e `fallthrough_exit`. Os três últimos campos são IDs. A saída por retorno recebe caminhos encerrados por `return`. A saída pelo fim recebe caminhos que terminam o corpo sem retornar. Criar uma saída artificial não a torna alcançável.

Use `CFG.terminate` para registrar o terminador e suas arestas. Cada aresta deve constar no conjunto de sucessores da origem e no conjunto de predecessores do destino.

Os destinos devem existir. Ao concluir a construção, os blocos alcançáveis devem possuir terminadores consistentes com suas conexões.

3.2 Continuação, escopos e chamadas

O percurso deve devolver o ID da continuação normal após um bloco da AST, ou `None` se nenhum caminho continua. Condicionais e laços devem respeitar esse contrato. Um `return` não possui sucessor normal para o próximo comando. Ramos encerrados não podem ser conectados à continuação da sequência.

`ScopeExit` preserva a fronteira de um escopo no percurso normal. Ela referencia o `Scope` produzido pela resolução de nomes e não acrescenta um novo nó sintático à AST. Atributos de escopo e de símbolo devem continuar referenciando os objetos da etapa anterior.

Construa um grafo separado por função. Uma chamada é uma operação do grafo do chamador. Não há aresta para o CFG da função chamada. Seus argumentos contêm leituras, mas a chamada não inicializa locais do chamador. A recursão não cria ciclos entre os grafos desta análise.

3.3 Exemplo: da AST ao CFG

O programa abaixo é válido. A Figura 1 mostra a AST e o CFG de *escolher*, incluindo dois blocos aninhados dentro do ramo verdadeiro.

```
int escolher(bool b) {
    int x;
    if (b) {
        {
            x = 10;
        }
    } else {
        return 0;
    }
    return x;
}

int main() {
    return escolher(true);
}
```

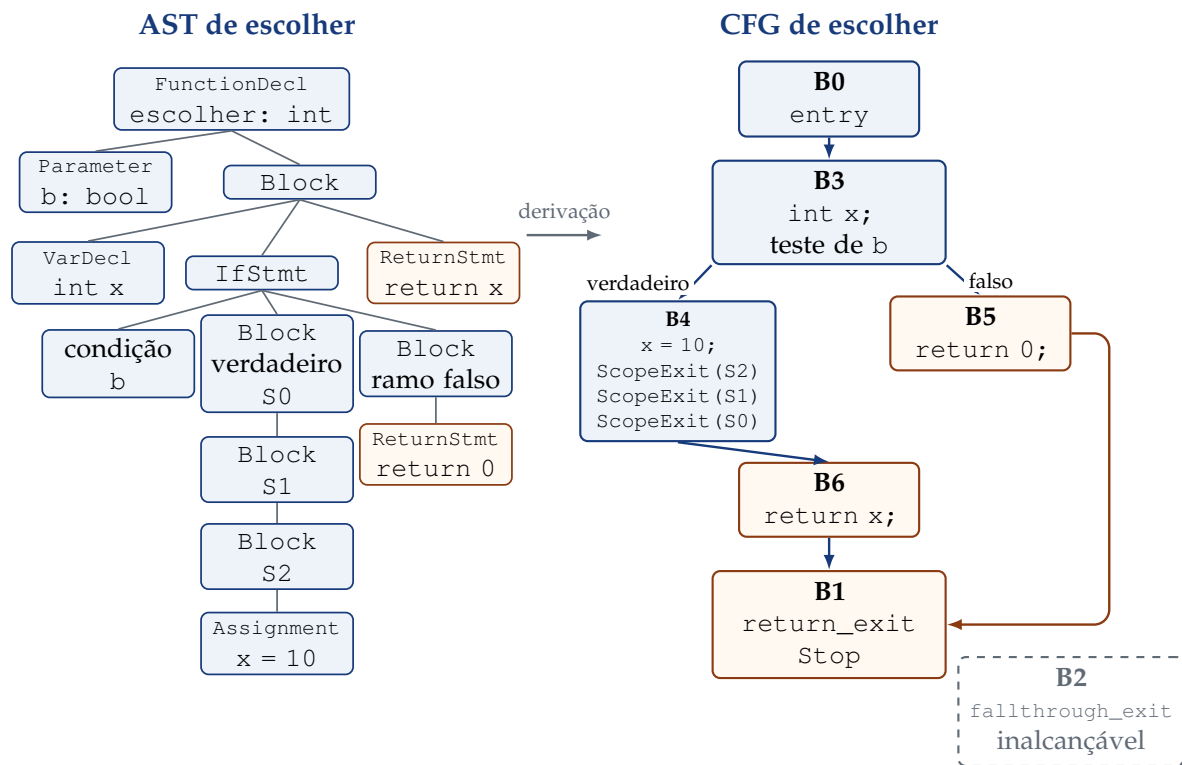


Figura 1: AST simplificada e CFG da mesma função. Linhas na árvore indicam composição. Setas no grafo indicam possíveis passos de execução.

Os escopos S1 e S2 pertencem aos dois blocos aninhados. Suas operações e saídas de escopo aparecem em B4. As chaves não exigem novos blocos básicos quando o fluxo permanece sequencial. As expressões são abreviadas. Os rótulos e a numeração são ilustrativos.

4 Alcance e inicialização definida

4.1 Alcance

Calcule o alcance antes de combinar estados. Somente predecessores alcançáveis participam da junção. Comandos após um retorno ou um laço sem saída normal não devem prolongar o caminho encerrado. A análise de fluxo não exige diagnóstico para código inalcançável e não verifica suas leituras.

A análise é conservadora. Condicionais mantêm os caminhos verdadeiro e falso, inclusive quando sua condição é um literal. Laços comuns mantêm o caminho que executa zero iterações. Somente `while (true)`, com o literal `true`, é reconhecido como laço sem saída normal. Não avalie outras expressões como constantes para eliminar caminhos.

4.2 Estados, transferência e ponto fixo

O estado é um conjunto de objetos `Symbol` definitivamente inicializados. Dois identificadores de mesmo nome podem representar símbolos diferentes. Parâmetros estão inicializados na entrada da função. Uma declaração com inicializador se torna inicializada depois da expressão. Uma atribuição inicializa seu destino depois da avaliação do lado direito.

Nas junções, sobreviverão somente fatos comuns a todos os caminhos que chegam àquele ponto. Em termos dos blocos alcançáveis:

$$\text{IN}(\text{entry}) = \{\text{símbolos dos parâmetros}\}$$

$$\text{IN}(B) = \bigcap_{P \in \text{pred}_{\text{alc}}(B)} \text{OUT}(P) \quad (B \neq \text{entry})$$

$$\text{OUT}(B) = T_B(\text{IN}(B))$$

A função de transferência T_B está fornecida. Ela reinicia uma declaração, acrescenta símbolos inicializados e remove os locais ao cruzar uma saída de escopo. A mesma declaração executada novamente em um laço deve começar sem valor garantido. Atribuições feitas a símbolos externos devem ser preservadas.

A análise de garantia em todos os caminhos (*must*) começa por uma aproximação superior nos demais blocos, usando o universo de símbolos da função. Recalcule os estados até que nenhum conjunto mude. Não limite a análise a uma única passagem ou a uma quantidade fixa de iterações. Calcular uma interseção não deve modificar os estados de seus predecessores.

4.3 Verificação das leituras

Emita diagnósticos somente depois da convergência. Percorra cada bloco alcançável na ordem de suas operações, começando por seu estado `IN`. Também verifique as expressões das condições e dos retornos. Uma atribuição posterior no mesmo bloco não justifica uma leitura anterior.

O destino de uma atribuição é uma escrita. Seu lado direito é uma leitura. O novo símbolo de uma declaração já está visível em seu próprio inicializador. Portanto, o

inicializador `int x = x;` lê o novo símbolo ainda não inicializado, mesmo se houver um `x` externo.

A verificação estática deve visitar os dois operandos de `&&` e `||`, inclusive quando um literal permitiria pular o segundo durante a execução. Por exemplo, uma leitura não inicializada no segundo operando de `false && b` deve ser diagnosticada. Essa escolha estática conservadora não altera a execução da MICROC.

Exemplo: leitura antes da atribuição

O programa seguinte contém uma leitura não inicializada em `print(x)`. A existência do símbolo é verificada na etapa anterior. Aqui, deve-se verificar se ele já recebeu valor em todos os caminhos até a leitura.

```
int main() {
    int x;
    print(x);
    return 0;
}
```

Na Figura 1, somente o ramo que atribui `x` alcança o bloco do retorno final. O ramo encerrado por `return 0` não participa da junção nesse ponto. Assim, a leitura em `return x` encontra o símbolo inicializado.

5 Fim das funções e diagnósticos

5.1 Retorno e divergência

Uma função `int` ou `bool` não deve alcançar `fallthrough_exit`. A presença de algum `return` não é suficiente. Uma função `void` pode chegar ao fim sem retorno explícito. A compatibilidade do valor de cada retorno com o tipo da função continua sendo verificada pela Semântica 1.

Um caminho pode divergir em `while (true)` sem alcançar o fim. Por isso, o programa abaixo é válido quanto ao retorno. Um laço com condição `!false` não recebe esse tratamento, pois a regra reconhece somente o literal `true`.

```
int main() {
    while (true) {}
}
```

5.2 Categorias e posição no código

As categorias anteriores permanecem válidas. Esta etapa acrescenta:

Categoria	Condição e posição
UNINITIALIZED_READ	Leitura sem inicialização garantida. Use o início do <code>IdentifierExpr</code> lido.
MISSING_RETURN	Fim alcançável de função não-void. Use o início da <code>FunctionDecl</code> .

Use `SemanticDiagnostic(kind, message, span)` e armazene todos os diagnósticos de fluxo antes de lançar `SemanticError`. Seu atributo `diagnostics` deve ser uma tupla não vazia. O texto da mensagem é escolha do grupo. Categoria e posição são avaliadas. Leituras distintas podem gerar erros distintos, mas as iterações do solver não devem duplicar o mesmo diagnóstico.

O runner escreve erros em `stderr`. Seu status é 0 para sucesso no modo solicitado, 1 para erro léxico, sintático ou semântico, 2 para erro de uso ou leitura de arquivo e 3 para um método do scaffold ainda não implementado.

6 Execução, testes e limites de implementação

Depois de recuperar as etapas anteriores em `previous/`, execute a partir da raiz do starter:

```
python -m pip install -r requirements-dev.txt
python runner.py test.mc --cfg-only
python runner.py examples/branches.mc --cfg-only
python runner.py examples/loop.mc --cfg-only
python runner.py examples/integrated.mc --dump-cfg
python -m pytest -q
```

`-cfg-only` executa lexer, parser, nomes e tipos e mostra o CFG. Esse modo não valida inicialização nem retorno ausente. A execução padrão valida todas as passagens. `-dump-cfg` mostra os grafos depois da validação completa. Não interprete o sucesso de `-cfg-only` como validade semântica do programa. As opções `-cfg-only` e `-dump-cfg` não podem ser usadas juntas.

Os testes de infraestrutura verificam partes já fornecidas pelo projeto:

```
python -m pytest -q tests/test_infrastructure.py
```

Os testes de CFG e fluxo representam a implementação completa e falham inicialmente nos métodos pendentes. Eles verificam casos centrais, integração, preservação da AST, categorias e posições de linha e coluna. Os testes de correção podem combinar as mesmas regras em outros programas, sem criar exigências novas.

O grupo pode criar auxiliares e reorganizar os algoritmos, preservando as interfaces e campos publicados. Não altere os contratos de tokens, AST, símbolos e diagnósticos. Implemente as análises em Python 3 sobre o CFG da AST validada. Não procure nomes no texto-fonte nem execute a análise semântica dentro do parser. Não substitua o CFG por uma nova linguagem intermediária.

Vivacidade, otimizações e geração de código não são exigidas nesta entrega. O CFG deve servir às verificações de inicialização e de alcance do fim.

7 Critérios de conclusão

Antes da entrega, confira se:

- o trabalho anterior completo está em `previous/`;
- condicionais e laços produzem caminhos e continuações corretos;
- as arestas e os terminadores do CFG são consistentes;

- o alcance ignora caminhos sem conexão com a entrada;
- a junção considera somente predecessores alcançáveis;
- os estados convergem e respeitam parâmetros, escopos e shadowing;
- leituras são verificadas em ordem, inclusive nos terminadores;
- funções não-void não alcançam o fim do corpo;
- os diagnósticos possuem categoria e posição sem duplicação;
- a AST conserva sua sintaxe, seus vínculos e seus tipos;
- todos os métodos pendentes foram implementados e os testes públicos passam; e
- o último commit está no repositório correto e foi conferido no GitHub Actions.

8 Referências para consulta

Consulte a especificação normativa da MICROC. Retome a Aula 9 para CFG, atribuição definida e alcance das saídas. A Aula 7 apresenta o próprio inicializador e a identidade dos símbolos.